

bundles, including this one, so try that out.

Given below are the Syntastic settings I have added in my `_vimrc`. You might have to change some of them, depending on your settings.

```
"Syntastic Settings
set statusline+=%#warningsmsg#
set statusline+=%(SyntasticStatuslineFlag())
set statusline+=%"
let g:syntastic_check_on_open=1
let g:syntastic_enable_balloons=1
let g:syntastic_always_populate_loc_list=1
let g:syntastic_echo_current_error=1
let g:syntastic_enable_highlighting=1
let g:syntastic_auto_jump=1
let g:syntastic_auto_loc_list=1
let g:syntastic_loc_list_height=5
let g:syntastic_perl_checkers=['perl', 'perlcritic']
let g:syntastic_id_checkers=1
let g:syntastic_python_checkers=['python', 'pyllakes']
"Enable below in case you want the error line highlighted
highlight SyntasticErrorLine guibg=#f06968
highlight SyntasticWarningLine guibg=#f06968
"Syntastic Error Signs"
let g:syntastic_error_symbol="E"
let g:syntastic_warning_symbol="W"
"End of Syntastic Settings
```

NERDTree: This plug-in also has been written by 'Scroolouse'. It gives a nice tree-like view of the files you are in. Install it using Vundle as follows: *BundleInstall 'scroolouse/nerdtree'*. Once done, add the following settings in the `_vimrc`.

```
"NERDTree Settings
"Open gVim with NERDTree
autocmd vimmer * if !argc() | NERDTree | endif
"Close NERDTree if the last file is closed.
autocmd bufer * if !winnr("$") == 1 &&
exists("b:NERDTreeType") && b:NERDTreeType == "primary" | q
| endif
"Give a nice look to NERDTree
let NERDChristmasTree = 1
"Show nice arrows instead of +
scriptencoding utf-8
set encoding=utf-8
let NERDTreeDirArrows = 1
```

Check out the screenshots in Figure 1. Just save the script file with a `'w'` in the command prompt and the errors/warnings will show up.

NeoComplCache: This plug-in has been written by 'Shougo'. This has the 'autocompletes as you type' feature to gVim, giving that final touch to the IDE-like behaviour.

Type *BundleInstall 'Shougo/neoconplcache.vim'* in the command mode and you are done! I have disabled the auto turn-on for the NeoComplCache plug-in so it does not kick off right after gVim loads. Type *NeoComplCacheEnable* at the gVim command prompt and then you can start using it. Alternatively, change the *set Neoconplcache_enable_at_startup* to 1 instead of 0.

PerlOmni: This plug-in has been written by Pedro. If you are into Perl Scripting, the PerlOmni plug-in will be quite helpful. Search *GitHub* and do a *BundleInstall* to get it installed; otherwise, just disable the *perlomni* settings (the last two lines in the code snippet below) if you don't install it.

The `_vimrc` settings given below are for both NeoComplCache and PerlOmni.

```
"NeoComplCache settings.
let g:acp_enableAtStartup = 0
" Use neoconplcache.
let g:neoconplcache_enable_at_startup = 0
" Use smartcase.
let g:neoconplcache_enable_smart_case = 1
" Set minimum syntax keyword length.
let g:neoconplcache_min_syntax_length = 3
let g:neoconplcache_lock_buffer_name_pattern = '\*ku\*'
" Define keyword.
if !exists('g:neoconplcache_keyword_patterns')
let g:neoconplcache_keyword_patterns = {}
endif
let g:neoconplcache_keyword_patterns['default'] = '\h\w\*'
" Recommended key-mappings.
" <CR>: close popup and save indent.
inoremap <silent> <CR> <C-R>=<SID>my_cr_function()<CR>
function! s:my_cr_function()
return neoconplcache#smart_close_popup() . "\<CR>"
" For no inserting <CR> key.
"return pumvisible() ? neoconplcache#close_popup() :
"\<CR>"
endfunction
" <TAB>: completion.
inoremap <expr><TAB> pumvisible() ? "\<C-N>" : "\<TAB>"
" <C-h>, <BS>: close popup and delete backward char.
inoremap <expr><C-h> neoconplcache#smart_close_popup()."\<C-h>"
inoremap <expr><BS> neoconplcache#smart_close_popup()."\<C-h>"
inoremap <expr><C-y> neoconplcache#close_popup()
inoremap <expr><C-e> neoconplcache#cancel_popup()
" Close popup by <Space>.
"inoremap <expr><Space> pumvisible() ? neoconplcache#close_popup() : "\<Space>"
```

```
" Enable omni completion.
autocmd FileType css setlocal omnifunc=cscscomplete#CompleteCSS
```